VibeCheck

会自我验收的 AI 应用生成器 · 生成不是终点，通过质量门禁才交付

pxx · 测试开发工程师
2026-11-12
在线 Demo：expulsion-lifting-urging.ngrok-free.dev
GitHub：github.com/xxstar2216/vibecheck

一句话

输入一句话，Agent 先生成可执行验收标准，再生成单文件 Web 应用，在隔离沙箱里逐条验收；失败自动修复一次，通过质量门禁才交付。

为什么做

AI 生成应用看起来能用，但常有运行时错误、刷新丢数据、改完不知道有没有回归；传统测试写不过来。VibeCheck 把验收标准前置：**先定义什么叫“做好了”，再让 AI 写代码。**对应 JD 的“让每一次生成更可靠、每一次发布更安全”。

核心流程

描述 → Spec Agent 规格 + 4-6 条验收 → Code Agent 流式单文件 HTML → 沙箱验收 点击/输入/DOM/存储/运行时/A11y → 质量门禁 ≥80 且零运行时错误 → 修复 自动 1 次 / 一键继续 → 交付 HTML / 报告 / Playwright

100	76s	7/7	1×
3 个模板满分	真实模型端到端	验收项通过	自动修复后放行

质量门禁

维度	权重	检查
运行时	30	未捕获异常、Promise rejection、console.error
验收通过率	40	由需求生成的 4-6 条可执行用例
可访问性	15	表单可访问名、图片 alt、按钮可访问名、html lang
韧性	15	完整文档、viewport、testid、存储桥、响应式、空状态、体积、外部依赖

放行：总分 ≥ 80 且零运行时错误。运行时 + 验收占 70%，因为它们最直接影响“能不能用”；A11y 与韧性用于拉开质量差距。

关键取舍

- **单文件 HTML**：换取秒级预览和零构建沙箱；代价是不支持多文件工程。
- **不透明源 sandbox + postMessage 存储桥**：生成代码读不到父页面；代价是多一次消息往返。
- **浏览器内 runner**：零基础设施、秒级反馈；可信度低于 Playwright，所以用例可导出为 `.spec.ts`。
- **IndexedDB**：无需登录、零数据库成本；不跨设备。
- **自动修 1 轮 + 编辑回归**：控制成本；复杂失败可一键继续修复。

完成度 / 下一步

已完成

- 生成 / 编辑 / 版本 / 回滚
- Spec → Code → Test → Repair 完整 Trace
- 四类质量检查、评分、放行/阻止
- 失败自动修复 + 一键继续修复
- 报告摘要复制；HTML / 报告 / Playwright 导出
- 沙箱隔离 + 存储桥 + 刷新持久化；3 个模板

限制 / 下一步

- 限制：单文件、浏览器 runner、只自动修 1 轮、内存限流、无登录/分享
- P0：20 条 golden prompt 评估集；Playwright 接 CI
- P1：失败版本 diff / 分享报告 / 多文件 WebContainer
- P2：视觉回归、模型 A/B、一键部署、线上遥测

AI 协作

Claude Code + DeepSeek API 用于脚手架、组件、提示词初稿和测试脚本；**关键取舍由我确定。**否决了：加 `allow-same-origin`、做登录/数据库、生成多文件工程、无限修复。手动修的关键问题：opaque origin 下 localStorage 不可用（存储桥）、流式预览抖动、验收等 `load` 被外部资源卡住（改等 `DOMContentLoaded` + 3s 上限）。

提交信息：在线 Demo https://expulsion-lifting-urging.ngrok-free.dev（首次打开点 Visit Site）
GitHub https://github.com/xxstar2216/vibecheck · 更详细取舍见仓库 `docs/DESIGN.md`